

LED Display System

Hourglass Animation using LED Multiplexing

1 Introduction

This project presents the design and implementation of an LED Display System that simulates an hourglass animation using simple LED multiplexing. The display consists of red LEDs arranged in an 8×8 grid on a perfboard and controlled using an Arduino Nano microcontroller. The project focuses on controlling multiple LEDs efficiently using shift registers and row-column scanning techniques. By rapidly switching rows and columns, different patterns can be displayed smoothly and continuously.

2 Circuit Overview

The circuit uses 74HC595 shift registers to expand the output capability of the Arduino, allowing control of multiple LEDs through serial communication. Transistors are used for row and column switching, while current-limiting resistors protect the LEDs from excess current. A potentiometer and push button are included for user interaction and display control. The tilt ball switch is used to detect the orientation of the display and trigger the hourglass animation.

3 Project Objectives

The main objective of this project is to demonstrate efficient LED multiplexing and display control through animated LED patterns. The important goals of the project are:

- Design and assemble an LED display on a perfboard
- Interface 74HC595 shift registers with Arduino Nano
- Implement row-column multiplexing for LED control
- Program the hourglass animation using timing techniques
- Add user controls using a potentiometer and push button
- Understand practical embedded system design concepts

4 Significance

This project demonstrates how LEDs can be controlled in a fast and efficient manner to generate smooth display patterns. The project also provides practical exposure to:

- Embedded programming
- Multiplexing techniques
- Timing control
- Shift register interfacing
- Transistor switching
- Digital electronics design

5 Components Used and Hardware Specifications

The Digital Hourglass Timer is designed using a low-power embedded architecture based on the Arduino Nano and a multiplexed LED matrix display. The system combines shift registers, transistor drivers, gravity sensing, and rechargeable battery management to create a portable interactive timer.

5.1 Microcontroller Unit

- Arduino Nano based on the ATmega328P 8-bit microcontroller.
- Operates at 16 MHz with 5V logic.
- Responsible for: LED matrix scanning, Shift register communication, Tilt detection, Potentiometer reading, Timer calculations, Buzzer control.

5.2 LED Matrix Display

- Total LEDs used: 64 discrete red LEDs
- Arranged in a custom hourglass geometry across 12 rows. Row Widths = [8,8,7,5,3,1,1,3,5,7,8,8]

5.3 Shift Register Display Driver

- Three 74HC595 shift registers are daisy-chained.
- Provides 24 parallel outputs using only: Data Pin, Clock Pin, Latch Pin.
- Greatly reduces Arduino GPIO usage.

5.4 Row Driver Stage

- 12 BC548 NPN transistors are used as low-side row switches.
- Only one row is activated at a time during multiplex scanning.

- This reduces instantaneous current demand and protects the shift registers from overload.

5.5 Sensors and Inputs

- **SW-520D Tilt Ball Switch:** Detects gravity inversion of the hourglass. Used to restart the sand animation when flipped.
- **Potentiometer:** Allows adjustable timer selection. Connected to analog pin A6.
- **Push Button:** Used for timer start/reset operations.

5.6 Audio Feedback

- A 5V active magnetic buzzer is used for sound alerts.
- Driven through a BC548 transistor for proper current amplification.

5.7 Power Supply Architecture

- **Power Source:** 18650 Lithium-Ion rechargeable battery.
- **Charging Module:** TP4056 charger module. Provides constant-current/constant-voltage charging and includes low-voltage cutoff protection.
- **Voltage Regulation:** MT3608 boost converter. Steps battery voltage to regulated 5V output.
- The regulated 5V output is connected directly to the Arduino Nano 5V pin, bypassing the onboard linear regulator to improve efficiency.

5.8 Resistors Used

- $8 \times 470 \Omega$ resistors for LED current limiting.
- $13 \times 1k \Omega$ resistors for transistor base protection.

6 Working Principle of the Project

The Digital Hourglass Timer operates using multiplexed LED scanning, gravity sensing, and software-controlled particle simulation to imitate the behavior of a real sand hourglass.

6.1 LED Matrix Operation

The 64 LEDs are arranged in a custom hourglass-shaped matrix. Since directly controlling all LEDs individually would require a large number of microcontroller pins, the system uses multiplexing with shift registers.

- The 74HC595 shift registers control the LED columns.
- The BC548 transistors control the rows.
- Only one row is activated at a time.

The Arduino rapidly scans through all 12 rows sequentially:

1. Column data for a row is shifted into the 74HC595 registers.
2. The corresponding BC548 transistor is turned ON.
3. LEDs in that row illuminate according to the shifted data.
4. The row is turned OFF.
5. The process repeats for the next row.

Because the scanning occurs extremely fast, persistence of vision makes the entire display appear continuously illuminated.

6.2 Multiplexing Duty Cycle

Since only one row is active at a time:

$$\text{Duty Cycle} = \frac{1}{12} \approx 8.3\%$$

Each row remains active for approximately 100 microseconds before the next row is scanned.

6.3 LED Current Calculation

To safely operate the LEDs and protect the shift registers, current-limiting resistors are used.

$$R = \frac{V_{cc} - V_f}{I_{LED}}$$

Where: $V_{cc} = 5\text{V}$, $V_f \approx 2\text{V}$, $I_{LED} = 6.4 \text{ mA}$

$$R = \frac{5 - 2}{0.0064} = 468.75 \Omega$$

A standard resistor value of 470Ω is selected.

6.4 Maximum Current Analysis

The widest rows contain 8 LEDs.

$$I_{row(max)} = 8 \times 6.4 \text{ mA} = 51.2 \text{ mA}$$

This current is safely handled by the BC548 transistor while remaining below the 74HC595 maximum package current rating.

6.5 Transistor Switching Operation

The BC548 transistors operate in saturation mode as digital switches. Base resistor calculation:

$$I_{base} = \frac{V_{out} - V_{be}}{R_b}$$
$$I_{base} = \frac{5 - 0.7}{1000} = 4.3 \text{ mA}$$

This base current is sufficient to fully switch the transistor ON and sink the row current safely.

6.6 Gravity Detection System

The SW-520D tilt switch detects physical inversion of the hourglass.

- When the device is flipped, the Arduino detects a logic change.
- The LED particle positions are inverted in software.
- The timer restarts automatically.

To avoid false triggering due to mechanical bouncing of the metal balls inside the tilt switch, a software debounce delay of 250 ms is implemented.

6.7 Timer Selection and Buzzer Feedback

The potentiometer connected to analog pin A6 selects the timer duration. Since the potentiometer was wired in reverse on the perfboard, the software corrects this using:

$$1023 - \text{analogRead}(A6)$$

The buzzer provides user feedback:

- Short 50 ms chirp during gravity flip.
- Continuous 5-second alarm after timer completion.

6.8 Power Efficiency

The use of multiplexing significantly reduces average power consumption. Average LED current:

$$I_{avg} = 6.4 \times \frac{1}{12} \approx 0.53 \text{ mA}$$

Maximum instantaneous LED power consumption:

$$P = 8 \times (2V \times 6.4 \text{ mA}) = 102.4 \text{ mW}$$

Thus, the project achieves excellent battery life while maintaining sufficient LED brightness.

7 Potentiometer — Timer Duration Selection

7.1 Overview

The potentiometer serves as the primary user control in this project. It is a variable resistor whose knob the user turns to select how long the hourglass timer should run before the alarm goes off. It is wired to analog pin A6 on the Arduino Nano, which reads the voltage coming off the wiper terminal and converts it into a number the firmware can work with. This conversion is handled by the Arduino's built-in 10-bit Analog to Digital Converter, which always produces a whole number somewhere between 0 and 1023 depending on how far the knob has been turned.

7.2 Working Principle

The component has three physical terminals. The leftmost pin is connected to the 5V supply, the rightmost pin is connected to ground, and the middle pin, called the wiper, is what goes to the Arduino. Internally, there is a strip of resistive material running between the two outer pins, and the wiper is a sliding contact that can sit anywhere along that strip. When the knob is turned all the way to one side, the wiper sits right at the ground end and the voltage it outputs is nearly zero. When turned all the way to the other side, the wiper is right at the 5V end and the output voltage is close to the full supply. At any position in between, the voltage is proportional to how far along the strip the wiper has moved. This relationship is described by the following equation:

$$V_{wiper} = V_{cc} \times \frac{R_{bottom}}{R_{total}}$$

and therefore

$$\text{ADC value} = \frac{V_{wiper}}{V_{cc}} \times 1023$$

So turning the knob fully counter-clockwise produces an ADC reading of 0, and fully clockwise produces a reading of 1023, with every possible position in between giving a proportional value.

7.3 The 7-Zone Timer Mapping

The firmware divides the full 0 to 1023 range into seven roughly equal zones, each about 146 units wide. Every zone corresponds to a different timer duration. When the user presses the button, the firmware reads the current ADC value, checks which zone it falls into, and sets a variable called `spawnRate` which controls how often one grain of sand is released from the top of the hourglass.

Since the hourglass always holds exactly 32 grains of sand, the time between spawning each grain must equal the total desired duration divided by 32.

$$\text{spawnRate} = \frac{\text{Total Duration in milliseconds}}{32}$$

ADC Range	Duration	spawnRate (ms)	Calculation
0 to 145	30 Seconds	937	$30000 \div 32 \approx 937$
146 to 291	1 Minute	1875	$60000 \div 32 = 1875$
292 to 437	2 Minutes	3750	$120000 \div 32 = 3750$
438 to 584	15 Minutes	28125	$900000 \div 32 = 28125$
585 to 730	30 Minutes	56250	$1800000 \div 32 = 56250$
731 to 876	45 Minutes	84375	$2700000 \div 32 = 84375$
877 to 1023	60 Minutes	112500	$3600000 \div 32 = 112500$

7.4 Firmware Logic

One important design decision is that the potentiometer is not read continuously while the timer is running. It is sampled only at the exact moment the user presses the button. This means the user must first turn the knob to the desired position and then press the button to confirm and start the timer. If they adjust the knob after pressing the button, nothing changes until the button is pressed again. Once a new duration is latched in, the firmware resets all the sand back to the top, clears the falling grain state, and marks the alarm as not yet played so it can fire again at the end of the new countdown.

8 Tilt Switch — Flip Detection and Gravity Reversal

8.1 Overview

The tilt switch is the component that makes this project feel like a real hourglass rather than just a countdown timer. When the user physically picks up the board and flips it upside down, the tilt switch detects the change in orientation and tells the firmware to reverse gravity. The sand that had collected at the bottom instantly becomes the new top supply, and the timer continues from that point in the new direction. The switch is connected to digital pin 5 of the Arduino Nano, which is configured with the internal pull-up resistor enabled so that no external resistor is needed in the circuit.

8.2 Working Principle

A tilt switch is a simple mechanical component. Inside the small cylindrical housing there is a tiny metal ball sitting inside a conductive tube. When the device is held in one orientation the ball rolls down to the bottom of the tube and makes contact with two metal pins, completing the circuit and pulling the pin reading to LOW. When the device is flipped, the ball rolls away from those pins, the circuit opens, and the pin reading returns to HIGH because of the internal pull-up resistor.

Physical Orientation	Pin Reading	isFlipped State
Normal upright position	HIGH	false
Flipped inverted position	LOW	true

8.3 The Debouncing Problem

Any mechanical switch produces a brief period of electrical noise when its state changes. The metal ball bounces slightly as it rolls and settles, causing the pin to flicker rapidly between HIGH and LOW for several milliseconds. To prevent this, a 50 millisecond software debounce is implemented using the `millis()` function. The firmware records a timestamp every time the pin changes state but only acts on the change if the signal has remained stable for at least 50 milliseconds afterwards.

$$(\text{millis}() - \text{lastTiltChange}) > 50 \text{ ms}$$

8.4 What Happens on a Flip

Once a valid flip is confirmed, the firmware carries out three things in quick succession. First, it swaps the sand counts, meaning whatever number was stored in the top sand variable moves to the bottom sand variable and vice versa. Second, it sets the orientation flag which causes the display engine to read the LED row data in reverse order so the image on the matrix always appears visually upright. Third, it resets the alarm flag so the buzzer is allowed to fire again when the new countdown finishes. A short chirp tone also plays at the moment of the flip, giving the user audio confirmation.

8.5 How the Display Mirrors Itself

The visual flip is handled inside the display rendering function with a single piece of logic. Normally the function scans rows 0 through 11 in order from top to bottom. When the orientation flag is set to true, it instead reads row data starting from row 11 and working back to row 0. The entire orientation correction costs exactly one subtraction operation per row per frame, adding virtually no processing overhead.

9 Real World Applications

1. **Digital Clocks and Timers:** The hourglass animation behaves similarly to count-down timers used in clocks and timing systems.
2. **Advertising Displays:** LED display panels are commonly used in scrolling signboards at shops, stations, and public information systems.
3. **Wearable Electronics:** Rapid LED multiplexing is also used in wearable displays and portable electronic lighting systems.
4. **Industrial Display Panels:** LED displays are used in industries for status indication and operational monitoring.
5. **Gaming and Interactive Displays:** Basic LED displays are also used in handheld gaming systems and electronic display projects.
6. **Educational Applications:** This project serves as a practical learning platform for understanding shift registers, multiplexing, and embedded systems.
7. **Orientation-Based Systems:** The tilt ball switch demonstrates orientation sensing used in phones, vehicles, and security devices.

10 Design Calculations and Specifications

1. LED Resistor Calculation

The resistor value for each LED is calculated using Ohm's law:

$$R = \frac{V_{cc} - V_{LED}}{I_{LED}}$$

Where $V_{cc} = 5V$, $V_{LED} \approx 2V$, $I_{LED} = 7mA$.

$$R = \frac{5 - 2}{0.007} \approx 428 \Omega$$

Practical resistor values used are 330Ω for higher brightness or 470Ω for safer operation.

2. Multiplexing Duty Cycle

For a 12-row scanning system:

$$\text{Duty Cycle} = \frac{1}{12} \approx 8.3\%$$

3. Average LED Current

Due to multiplexing, the average LED current becomes:

$$I_{avg} = I_{peak} \times \text{Duty Cycle} = 8mA \times \frac{1}{12} \approx 0.67mA$$

4. Current Requirement Per Row

If all LEDs in one row turn ON simultaneously:

$$I_{row} = 8 \times I_{LED} = 8 \times 8\text{mA} = 64\text{mA}$$

Since the 74HC595 cannot safely sink this current directly, NPN transistors are used as row drivers.

5. 74HC595 Shift Register Specifications

Parameter	Typical Value
Operating Voltage	2V – 6V
Recommended Supply Voltage	5V
Maximum Current Per Pin	20mA (absolute maximum)
Recommended Safe Current	6mA – 8mA
Maximum Total Chip Current	70mA
Output Type	Serial-In Parallel-Out
Storage Register	8-bit
Clock Type	Edge Triggered

6. Transistor Switching Requirement

NPN transistors are used as low-side switches because the row current exceeds the shift register's capability. The transistor handles higher row current safely, protects the shift register, improves switching performance, and enables proper row multiplexing.

7. Power Consumption Estimate

Approximate power consumed by one LED:

$$P = V \times I = 2\text{V} \times 8\text{mA} = 16\text{mW}$$

11 Conclusion

This project successfully demonstrates the implementation of an LED Display System capable of generating an animated hourglass effect using LED multiplexing. The project combines hardware interfacing and embedded programming techniques to create smooth visual animations through rapid row and column scanning. The use of the Arduino Nano, shift registers, transistor switching, and LED control methods enabled efficient operation of the display while minimizing the number of required microcontroller pins.

The inclusion of the tilt ball switch, potentiometer, and push button improved the overall functionality and interactivity of the system. This project reflects the working principles behind modern display systems used in advertising boards, digital clocks, wearable electronics, and industrial display panels.

The project also opens opportunities for several future enhancements and improvements. The display system can be upgraded using dedicated LED driver ICs such as the MAX7219 to simplify multiplexing and improve brightness control. Higher resolution LED matrices and smoother animation algorithms can further enhance the visual quality of the hour-glass effect. Additional features such as wireless control, rechargeable battery monitoring, scrolling text display, sound effects, and programmable animations can also be integrated. The overall power efficiency and compactness of the hardware can be improved further through optimized PCB design and advanced display driving techniques.